

Product CRUD API Practice Implementation Guide

[Laravel 12 Framework Reference Manual](#)

This document provides the complete, production-ready codebase implementation matching all the strict criteria outlined within your assignment. The controller architecture leverages the robust `Validator::make()` manual facade framework, guaranteeing uniform validation responses, absolute data cast integrity, and reliable image storage cleanup pipelines.

1. Database Layer

The solution enforces a structured database constraint rule. The cascade flag prevents orphan table records when a parent relationship category is removed.

database/migrations/2026_06_11_000000_create_products_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            // Requirement A: ForeignKey referencing categories with cascade deletion
            $table->foreignId('category_id')->constrained('categories')-
            >onDelete('cascade');
            $table->string('name');
            $table->string('image')->nullable();
            $table->decimal('price', 10, 2);
            $table->integer('stock');
            $table->boolean('is_active')->default(true);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
```

```

{
    Schema::dropIfExists('products');
}
};

```

app/Models/Product.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Support\Facades\Storage;

class Product extends Model
{
    protected $fillable = [
        'category_id',
        'name',
        'image',
        'price',
        'stock',
        'is_active',
    ];

    /**
     * Requirement A: Enforce Attribute Casting ($casts)
     * Direct database values are parsed into actual JSON floats, integers, and booleans.
     */
    protected $casts = [
        'price' => 'float',
        'stock' => 'integer',
        'is_active' => 'boolean',
    ];

    /**
     * Requirement A: Append custom virtual field to the JSON string serialization
     */
    protected $appends = ['image_url'];

    /**
     * Requirement A: One-to-Many Inverse Link (belongsTo Relationship)
     */
    public function category(): BelongsTo
    {
        return $this->belongsTo(Category::class);
    }

    /**
     * Requirement A: Accessor for generating absolute external URLs for client
     applications
     */
    public function getImageUrlAttribute(): ?string
    {
        if (!$this->image) {

```

```

        return null;
    }
    return url(Storage::url($this->image));
}
}

```

2. API Routing Architecture

Standard resource endpoints are declared explicitly inside the web service configuration file using the standardized clean API Resource engine framework matrix syntax mapping rules.

routes/api.php

```

<?php

use App\Http\Controllers\Api\ProductController;
use Illuminate\Support\Facades\Route;

/**
 * Requirement B: Standard API Resource mapping syntax
 * Implicitly maps GET, POST, PUT, and DELETE HTTP verbs to core controller endpoints.
 */
Route::apiResource('products', ProductController::class);

```

3. Controller Layer (Manual Validator Blueprint)

The backend validation matrix acts directly inside the custom inline method lifecycle using the standard `Validator::make()` architecture framework to intercept anomalous input values.

app/Http/Controllers/Api/ProductController.php

```

<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Facades\Validator;

class ProductController extends Controller
{
    /**
     * Endpoint: GET /api/products
     * Fetch all products including embedded category details.
     */
    public function index(): JsonResponse
    {
        $products = Product::with('category')->get();
    }
}

```

```

        return response()->json([
            'success' => true,
            'data'     => $products
        ], 200);
    }

    /**
     * Endpoint: POST /api/products
     * Process multi-part payloads, validate manually, and store structural records.
     */
    public function store(Request $request): JsonResponse
    {
        // Requirement C: Explicit Input Validation Matrix via Validator::make()
        $validator = Validator::make($request->all(), [
            'category_id' => 'required|exists:categories,id',
            'name'         => 'required|string|max:255',
            'image'         => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
            'price'         => 'required|numeric|min:0',
            'stock'         => 'required|integer|min:0',
            'is_active'     => 'boolean',
        ]);

        // Graceful error pipeline triggering a 422 standard response structure
        if ($validator->fails()) {
            return response()->json([
                'success' => false,
                'message' => 'Validation parameters failed.',
                'errors'   => $validator->errors()
            ], 422);
        }

        $validated = $validator->validated();
        $validated['is_active'] = $request->boolean('is_active', true);

        // File Upload Pipeline
        if ($request->hasFile('image')) {
            $path = $request->file('image')->store('products', 'public');
            $validated['image'] = $path;
        }

        $product = Product::create($validated);
        $product->load('category');

        return response()->json([
            'success' => true,
            'data'     => $product
        ], 201);
    }

    /**
     * Endpoint: GET /api/products/{product}
     * Render data records for a standalone product alongside its parent category.
     */
    public function show(Product $product): JsonResponse
    {
        $product->load('category');
    }

```

```

        return response()->json([
            'success' => true,
            'data'     => $product
        ], 200);
    }

/**
 * Endpoint: POST /api/products/{product} (Using method spoofing via form-data)
 * Update target records and handle physical file replacements safely.
 */
public function update(Request $request, Product $product): JsonResponse
{
    $validator = Validator::make($request->all(), [
        'category_id' => 'required|exists:categories,id',
        'name'         => 'required|string|max:255',
        'image'         => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',
        'price'         => 'required|numeric|min:0',
        'stock'         => 'required|integer|min:0',
        'is_active'     => 'boolean',
    ]);

    if ($validator->fails()) {
        return response()->json([
            'success' => false,
            'message' => 'Validation parameters failed.',
            'errors'   => $validator->errors()
        ], 422);
    }

    $validated = $validator->validated();
    $validated['is_active'] = $request->boolean('is_active', $product->is_active);

    // File Replacement Pipeline
    if ($request->hasFile('image')) {
        // Delete old file asset from storage disk if a replacement exists
        if ($product->image && Storage::disk('public')->exists($product->image)) {
            Storage::disk('public')->delete($product->image);
        }

        $path = $request->file('image')->store('products', 'public');
        $validated['image'] = $path;
    }

    $product->update($validated);
    $product->load('category');

    return response()->json([
        'success' => true,
        'data'     => $product
    ], 200);
}

/**
 * Endpoint: DELETE /api/products/{product}
 * Safely drop files from local disk storage, then purge the row record.
 */

```

```

public function destroy(Product $product): JsonResponse
{
    if ($product->image && Storage::disk('public')->exists($product->image)) {
        Storage::disk('public')->delete($product->image);
    }

    $product->delete();

    return response()->json([
        'success' => true,
        'message' => 'Product and linked assets successfully removed.'
    ], 200);
}
}

```

4. Essential CLI Configurations

To successfully pass your grading rubric requirements, execute the following operational sequence inside your workspace terminal environment:

- **Step 1 (Database Migration Sync):** Run `php artisan migrate` to update and seed your structural tables.
- **Step 2 (Storage Symbolic Linking):** Initialize asset exposure by execution of `php artisan storage:link`. This addresses the virtual link required to resolve the absolute URL field generation.
- **Step 3 (Booting Framework Server Instance):** Launch local serving interfaces using `php artisan serve`.

Critical Testing Insight (Postman Method Spoofing)

PHP runtime engines are restricted natively from reading binary multi-part form payloads inside standard HTTP `PUT` or `PATCH` streams. When setting up Postman testing configurations for the update endpoint:

1. Configure the foundational Postman operational method setting to **POST**.
2. Pass your address target URL route path pointing explicitly to your specific item index (e.g., `/api/products/1`).
3. Inside your **form-data** parameter table rows, append an input variable key field named `_method` and set its plaintext parameter rule assignment value explicitly to `PUT`. This ensures clean structural execution workflows.